Daedalus system architecture

How the code works today and how the nodes communicate · ASP Project · `ros2_ws/src/daedalus_control` · PX4 SITL + Gazebo Harmonic + ROS 2 Humble + RTAB-Map · October 2026

Summary. The drone flies a pre-planned route through the warehouse while it builds a 3D map. `trajectory_generator` plans the route on a known occupancy grid (A*) and turns it into a smooth, time-parametrised trajectory. `offboard_control` is the only node that commands PX4: it converts each setpoint from ROS (ENU) to PX4 (NED), keeps the Offboard heartbeat alive, and switches mode and arms the drone. In parallel, **RTAB-Map** builds the map from the simulated ZED 2i colour and depth images, using PX4's EKF odometry (converted by `px4_odometry`) as its motion estimate.

1 · Overview

There are two data paths. In the **control loop**, setpoints flow from the planner to PX4, and PX4 drives the motors of the simulated drone. In the **perception path**, camera images from Gazebo and odometry from PX4 flow into RTAB-Map, which produces the map.

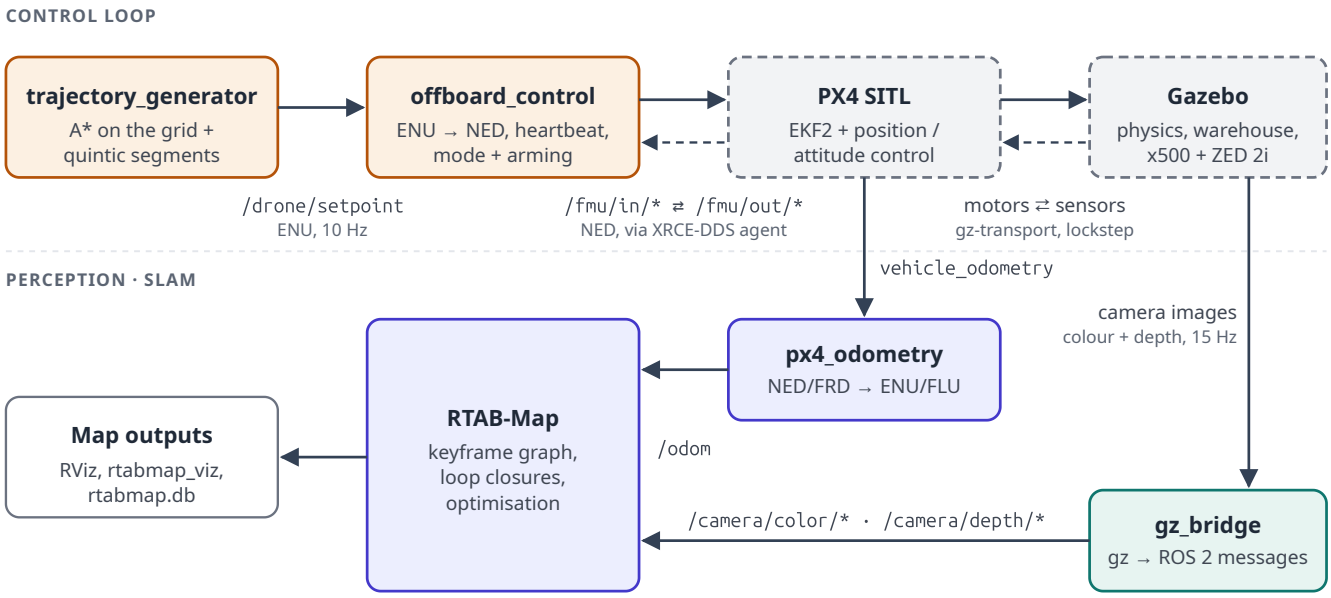


Figure 1. The two data paths. Solid arrows are commands or data; dashed arrows are feedback. Grey dashed boxes run outside ROS in the `sim` container. Colours: **flight control** **SLAM** **simulation bridge**.

Containers

Container	Image	What runs there
`sim`	`px4io/px4-sitl-gazebo:latest` (PX4 `main` @ `61907962`)	Gazebo Harmonic (headless server) with the warehouse world and the drone, and PX4 SITL with its uXRCE-DDS client. The world and the drone model are mounted from `gazebo/`.
`agent`	`warehouse-drone:humble`	Micro XRCE-DDS Agent on UDP port 8888. It turns PX4's uORB topics into ROS 2 topics `/fmu/out/*`, and `/fmu/in/*` back into uORB.
`dev`	`warehouse-drone:humble` (ROS 2 Humble, `px4_msgs` pinned to `57f88be`, RTAB-Map)	All ROS 2 nodes in `ros2_ws`. `/ws/images` is mounted on the host as `<project>/images` (e.g. to copy `rtabmap.db` out).

All containers use host networking. The Gazebo GUI, when used, runs on the host (`scripts/start.sh`). Every ROS node uses simulation time (`/clock` from Gazebo), so images, odometry and TF share one clock.

2 · ROS 2 node graph

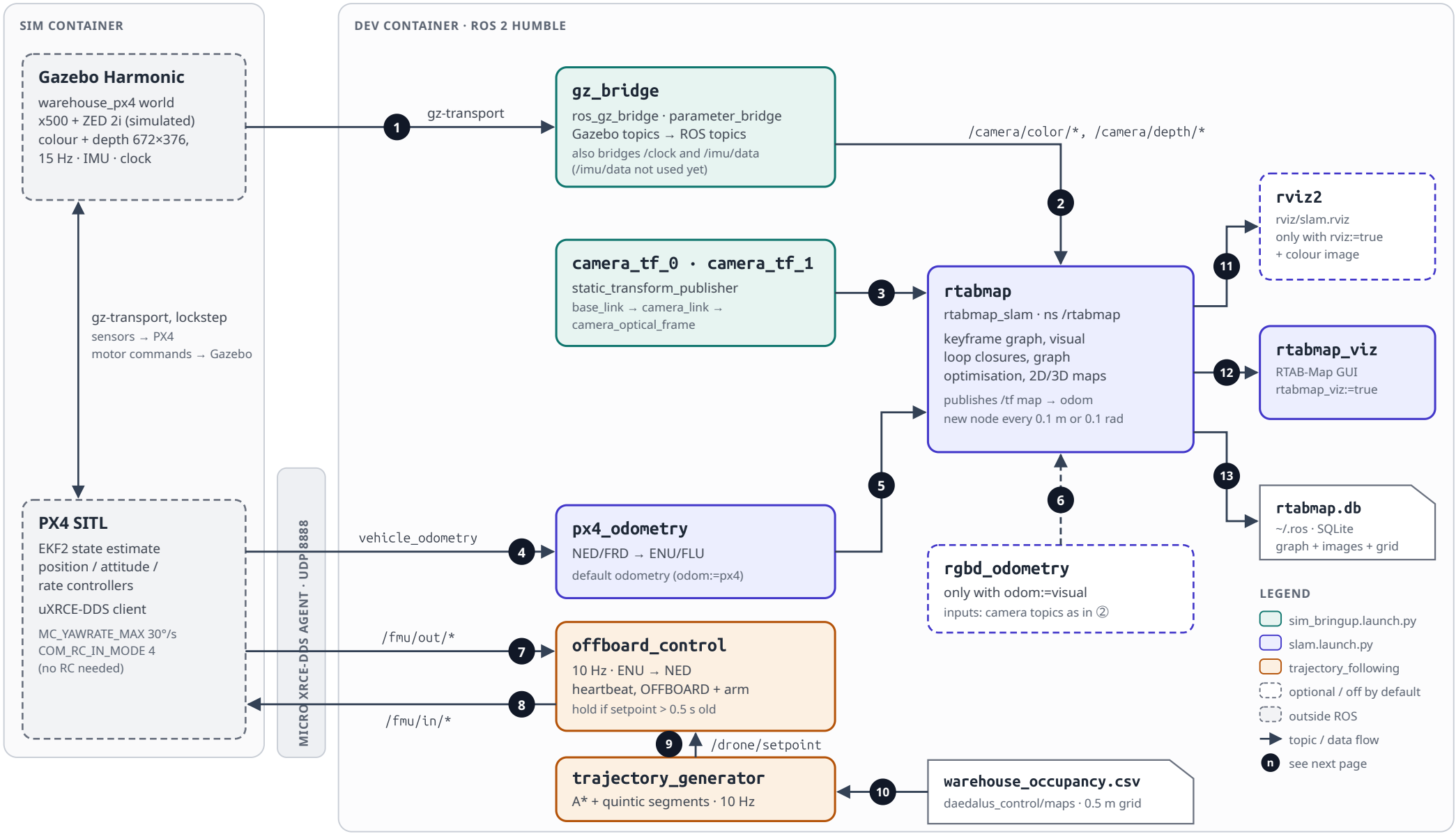


Figure 2. Every process started by `trajectory_following.launch.py slam:=true`, and the topics between them. Numbers refer to the table on the next page. Optional nodes (dashed) start only with the launch argument written in the box.

Connections in Figure 2

#	From → to	Topic(s)	Message type and notes
①	Gazebo → gz_bridge	`/world/warehouse_px4/model/x500_depth_0/link/…/sensor/…`, `/clock`	gz-transport (not ROS). Camera images, camera_info, depth point cloud, IMU and the simulation clock.
②	gz_bridge → rtabmap	`/camera/color/image_raw`, `/camera/color/camera_info`, `/camera/depth/image_raw`	`sensor_msgs/Image` (rgb8, 32FC1 metres), `CameraInfo`; frame `camera_optical_frame`; 15 Hz, approximately synchronised by RTAB-Map.
③	camera_tf → all	`/tf_static`	`base_link → camera_link` (0.135, 0.06, 0.256 m) and `camera_link → camera_optical_frame` (roll −90°, yaw −90°).
④	PX4 → px4_odometry	`/fmu/out/vehicle_odometry`	`px4_msgs/VehicleOdometry`, NED position, FRD body, best-effort QoS.
⑤	px4_odometry → rtabmap	`/odom`, `/tf`	`nav_msgs/Odometry` (ENU/FLU, covariance from the EKF variances) and TF `odom → base_link`.
⑥	rgbd_odometry → rtabmap	`/odom`, `/tf`	Same interface as ⑤, from visual odometry. Only with `odom:=visual`; resets itself after being lost.
⑦	PX4 → offboard_control	`/fmu/out/vehicle_local_position_v1`, `/fmu/out/vehicle_status_v4`	Current NED position (validity flags) and the navigation / arming state.
⑧	offboard_control → PX4	`/fmu/in/offboard_control_mode`, `/fmu/in/trajectory_setpoint`, `/fmu/in/vehicle_command`	Heartbeat (position control) + setpoint at 10 Hz; mode switch and arm commands when needed.
⑨	trajectory_generator → offboard_control	`/drone/setpoint`	`daedalus_interfaces/FlightSetpoint`: ENU position, velocity, acceleration, yaw, yaw rate; 10 Hz.
⑩	file → trajectory_generator	`maps/warehouse_occupancy.csv`	Read once at start-up. Generated from the world by `scripts/gen_occupancy_grid.py`.
⑪	rtabmap → rviz2	`/rtabmap/cloud_map`, `/rtabmap/grid_prob_map`, `/rtabmap/mapPath`, `/tf`	3D map (PointCloud2), 2D occupancy grid, optimised trajectory, and TF `map → odom`.
⑫	rtabmap → rtabmap_viz	`/rtabmap/mapData`, `/rtabmap/info`	Graph, keyframes and loop-closure statistics for RTAB-Map's own GUI.
⑬	rtabmap → file	`~/.ros/rtabmap.db`	SQLite database inside the `dev` container; wiped at start unless `new_map:=false`. Copy it to `/ws/images` to keep it.

3 · Flight control

trajectory_generator

It runs open-loop: it does not read the drone's position, it assumes the drone starts on the ground at the spawn point. On its first timer tick (10 Hz) it computes the whole trajectory, then it publishes one sample per tick.

1. **Map.** Loads the 0.5 m occupancy grid, computes the clearance of every cell, and inflates obstacles by 4 cells (2 m).
2. **Route.** Vertical take-off to the first target's height, then A* between consecutive targets of `target_pos` (4-connected; cost = length + 2 per turn + a centring term `center_weight = 0.3` per metre closer than 4.5 m to an obstacle). The path is compacted to its corners.
3. **Timing.** Each straight segment becomes a quintic polynomial with zero velocity and acceleration at both ends, its duration set by `v_max = 1 m/s` and `a_max = 1 m/s²`. Yaw is the heading of the segment.
4. **Output.** Samples every 0.1 s on `/drone/setpoint`; after the last one it repeats the final setpoint.

offboard_control

The only node that talks to PX4. Every 100 ms it sends `OffboardControlMode` + `TrajectorySetpoint`, converting ENU to NED ($x_{NED} = y_{ENU}$, $y_{NED} = x_{ENU}$, $z_{NED} = -z_{ENU}$, $\psi_{NED} = \pi/2 - \psi_{ENU}$), and runs this start-up sequence:

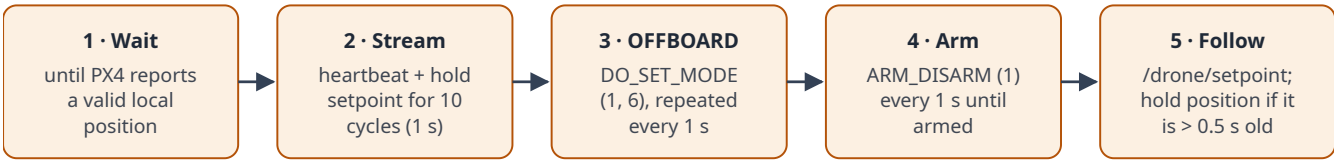


Figure 3. `offboard_control` start-up. Steps 2–5 overlap in time: the heartbeat never stops, because PX4 leaves Offboard mode if it does.

Coordinates

Targets in `target_pos` are ENU, in metres, **relative to the spawn point** (world −10.2, −25). Lane centres at x = 0.0, 7.6 and 16.2; cross corridors at y = −4.5 and 54.5. PX4 works in NED with the same origin. With `odom:=px4`, RTAB-Map's `map` frame uses these same coordinates, so a target and the map can be compared directly.

4 · SLAM

RTAB-Map receives synchronised colour + depth frames, an odometry estimate and the camera TF. Every 0.1 m or 0.1 rad of motion it adds a keyframe (node) to a pose graph. It looks for **loop closures** with a bag-of-words search over the images, verifies them geometrically, and re-optimises the graph. The correction appears as the TF `map → odom`. Maps are rebuilt from the optimised keyframes.

Odometry source (launch argument `odom`):

- `px4` (default) — `px4_odometry` converts PX4's EKF odometry (IMU + GPS in SITL). It never gets lost, so the map keeps growing in front of bare walls and through turns.
- `visual` — RTAB-Map's `rgbd_odometry` (features + PnP). It is lost in front of untextured walls; with `Odom/ResetCountdown = 1` it restarts, and RTAB-Map begins a new map session that is merged at the next loop closure.



Arrows point from parent to child; the label is the node that publishes the transform.

Figure 4. TF tree. `camera_optical_frame` is the `frame_id` of all camera messages (z forward, x right, y down).

Outputs. Live: `/rtabmap/cloud_map`, `/rtabmap/grid_prob_map`, `/rtabmap/mapPath` (RViz, `rviz/slam.rviz`) and rtabmap_viz. Offline: `rtabmap.db`, which `rtabmap-export` turns into a point cloud (`.ply`) or a textured mesh (`.obj`), and `rtabmap-databaseViewer` opens for inspection.

5 · Launch files

File	Starts	Arguments (default)
`trajectory_following.launch.py`	trajectory_generator, offboard_control; includes the two files below	`bridge` (true), `slam` (false), `odom` (px4), `rviz` (false), `rtabmap_viz` (true)
`sim_bringup.launch.py`	gz_bridge, camera_tf_0/1, optional RViz point-cloud view	`camera` (false; true when included above), `pointcloud` (false), `world`, `model`
`slam.launch.py`	px4_odometry or rgbd_odometry, rtabmap, rtabmap_viz, rviz2	`odom` (px4), `new_map` (true), `localization` (false), `rtabmap_viz` (true), `rviz` (false), `bridge` (true; false when included above)

Typical run, from the ROS shell opened by `scripts/start.sh`:

```
ros2 launch daedalus_control trajectory_following.launch.py slam:=true rviz:=true
```

6 · Notes and current limits

- `trajectory_generator` plans on the ground-truth grid, not on the SLAM map, and does not use feedback; PX4 tracks the setpoints.
- Yaw changes are not shaped by the generator: the heading jumps at each corner and PX4 limits the turn rate (`MC_YAWRATE_MAX` = 30°/s).
- There is no landing command: the route ends hovering at its last target, and a final target at z = 0 only brings the drone down by position, with PX4 still armed in Offboard mode.
- `/imu/data` is bridged but no node uses it yet.
- `px4_msgs` is pinned to commit `57f88be` to match the PX4 image; both must be updated together, otherwise the `/fmu/out/*_v4` topics stop arriving.